

A TECHNICAL REPORT

On

DIGITAL TEMPERATURE MONITORING AND FAN CONTROL SYSTEM

Prepared by

(GROUP 2)

Submitted to

**THE DEPARTMENT OF COMPUTER ENGINEERING,
SCHOOL OF ENGINEERING AND ENGINEERING TECHNOLOGY,
FEDERAL UNIVERSITY OF TECHNOLOGY, AKURE, ONDO STATE,
NIGERIA.**



In partial fulfillment of the requirements for the course

CPE 403: MICROPROCESSOR SYSTEM AND INTERFACING

SUPERVISED BY

DR. C.S. ODEYEMI

March, 2026.

GROUP MEMBERS

S/N	NAME	MATRIC NUMBER
1	FOLALU IDOWU PETER	CPE/23/4797
2	AJAYI ENIOLA VICTORIA	CPE/23/4775
3	OMOLOJU CHAMPION TEMILORUN	CPE/22/0028
4	AMEH GIDEON UKOLO-OJO	CPE/22/9993
5	AJAYI AYOMIDE OLUWATIMILEHIN	CPE/22/9987
6	IGBALAYE EBUNOLUWA KATHRYN	CPE/22/0011
7	ADEDOKUN MARTINS SIJUWADE	CPE/22/9979
8	OGWUCHE ISALAH OKOH	CPE/20/5617
9	BODUNDE BOLUWATIFE MICHAEL	CPE/22/0002
10	AYANO GIFT ADURAGBEMI	CPE/22/9997
11	ABOLADE SAMUEL AYOMIDE	CPE/22/9974
12	ADEKUNLE TOBILOBA HEPHZIBAH	CPE/22/9982
13	OBINEYE JEPHTAH OMOTAYO	CPE/22/0017
14	OKECHUKWU NKEMDIRIM	CPE/22/0021
15	MEBAWONDU AYOKARI	CPE/22/0015
16	OLATONA EXCELLENT	CPE/22/0024
17	SUULOLA OLUWAGBEMIGA FOLAHANMI	CPE/22/0032
18	OYEWALE PRUDENCE CHIDINMA	CPE/20/5631
19	AYINMIRO AYORINDE ISAAC	CPE/20/5595

Table of Contents

Chapter 1: Introduction	5
1.1 Project Background	5
1.2 Aim and Objectives of the Project	6
1.3 Scope of the Project	6
1.4 Organization of the Report	7
Chapter 2: System Design & Architecture	9
2.1 System Overview	9
2.2 Hardware Components and Description	9
2.2.1 The ESP8266 Microcontroller	9
2.2.2 DHT11 Temperature and Humidity Sensor	10
2.2.3 16x2 Liquid Crystal Display with I2C Interface	11
2.2.4 Direct Current (DC) Cooling Fan	11
2.2.5 BC547 NPN Transistor and Current Limiting Resistor	12
2.3 Microprocessor Architecture of the ESP8266	13
2.4 Software Design and Program Flow	13
2.4.1 Initialization (The setup() Function)	13
2.4.2 Operational Logic (The loop() Function)	14
2.4.3 System Program Flowchart	14
2.5 System Block Diagram	15
Chapter 3: Implementation, Testing, and Conclusion	16
3.1 System Implementation	16
3.1.1 Prototype Fabrication and Assembly	16
3.1.2 Sub-System Integration and Housing	16
3.2 Testing and Results	17
3.2.1 Test Methodology and Experimental Setup	17
3.2.2 Observation of Experimental Results	17
3.2.3 Analysis of As-Built Performance Flaw (Emitter-Follower)	17
3.3 Conclusion and Future Work	18
3.3.1 Conclusion	18
3.3.2 Future Work and Recommended Optimizations	20

Chapter 4: Appendix	21
4.1 Source Code	21

Chapter 1: Introduction

1.1 Project Background

In the contemporary era of automation and smart systems, the ability to monitor and regulate environmental parameters is a critical requirement across various domains, ranging from industrial manufacturing and server room maintenance to smart home environments and agricultural storage. Two of the most crucial environmental factors that dictate the stability and comfort of a closed system are temperature and relative humidity. Excessive heat can lead to the thermal throttling of electronic components, accelerated degradation of materials, and general system inefficiencies. Conversely, manual monitoring of these parameters is tedious, prone to human error, and lacks the rapid response time necessary to prevent heat-induced damage.

Historically, temperature regulation relied on analog thermostats or manual intervention. However, with the rapid advancement of embedded systems and micro-computing, there has been a significant paradigm shift toward intelligent, automated control systems. These modern systems utilize precise digital sensors to continuously poll the environment, feeding raw data into a central processing unit that executes predefined logical operations. By bridging physical environmental metrics with software-driven logic, it becomes possible to create reactive environments that self-regulate without the need for constant human oversight.

This project explores the design and implementation of such an automated system, focusing on localized thermal management. At the core of this hardware-software integration is a central microcontroller capable of high-speed data acquisition. When paired with a digital temperature and humidity sensor, the microcontroller can maintain a constant awareness of the ambient climate. The true value of this system, however, lies in its active response mechanism. Rather than merely logging data, the system is engineered to evaluate the current temperature against a pre-programmed threshold. When the environment crosses into a critical heat zone, the system autonomously actuates a direct current (DC) cooling fan via a solid-state transistor switching circuit.

Furthermore, to provide immediate, on-site feedback to users or system administrators, local display technologies are incorporated. This ensures that the current climatic conditions and the operational status of the cooling mechanism are fully transparent and accessible at a glance. By combining sensing, processing, actuation, and display into a

single, cohesive unit, this project demonstrates a fundamental application of control systems engineering and embedded hardware design.

1.2 Aim and Objectives of the Project

Aim

The primary aim of this project is to design, program, and implement an automated, microcontroller-based environmental monitoring system that dynamically triggers an active cooling mechanism when ambient temperatures exceed a predefined thermal threshold.

Objectives

- **Sensor Integration:** To successfully interface a DHT11 digital temperature and humidity sensor with an ESP8266 microcontroller, establishing reliable data communication for real-time environmental polling.
- **Logical Processing and Control:** To develop and flash a C++ based control algorithm onto the microcontroller that continuously reads sensor data and evaluates it against a hardcoded threshold of 33°C.
- **Actuator Circuit Design:** To construct an electronic switching circuit utilizing an NPN transistor (BC547) to safely drive a 5V DC cooling fan, isolating the high-current demands of the motor from the low-current logic pins of the microcontroller.
- **Local Data Visualization:** To integrate a 16x2 Liquid Crystal Display (LCD) equipped with an I2C communication backpack, allowing for the real-time, easily readable output of the current temperature (in Celsius) and relative humidity (as a percentage).
- **System Automation:** To synchronize all software and hardware components such that the fan actuation and display updates occur autonomously and continuously

1.3 Scope of the Project

This project focuses strictly on localized, standalone environmental monitoring and hardware actuation, intentionally omitting remote networking capabilities to prioritize system reliability and low latency. While the ESP8266 microcontroller possesses inherent Wi-Fi capabilities, this prototype does not connect to the internet, local area networks, or Internet of Things (IoT) cloud dashboards. The system operates entirely offline, making independent logical decisions based solely on real-time data acquired from its direct physical environment.

The thermal management capabilities are defined by the specific hardware components selected. The environmental data acquisition is limited by the operational constraints of

the DHT11 sensor, which provides an adequate temperature measurement range of 0°C to 50°C with an accuracy of $\pm 2^\circ\text{C}$, and humidity readings from 20% to 90% with an accuracy of $\pm 5\%$. While this is perfectly sufficient for indoor ambient monitoring, it is not designed for extreme industrial environments or high-precision laboratory applications.

Furthermore, the cooling mechanism is restricted to a small-scale 5V DC fan. Therefore, the scope of the physical cooling effect is localized. The project aims to demonstrate the control logic and switching architecture required to trigger a cooling state, rather than attempting to comprehensively cool a large room or structural environment. The logic is governed by a static, hardcoded threshold. Specifically, the firmware dictates that the cooling fan will only actuate when the ambient temperature reaches or exceeds 33°C, and will power down once the temperature drops below this absolute threshold. There is no dynamic threshold adjustment via user input after the firmware has been compiled and flashed to the board.

1.4 Organization of the Report

Chapter 1: Introduction - This initial chapter establishes the foundational context of the project. It outlines the overarching background of automated environmental control systems, clearly defines the primary aim and specific engineering objectives, and sets the boundaries of the prototype within the project scope.

Chapter 2: System Design & Architecture - This chapter delves into the core engineering aspects of the project. It details the overall system architecture, exploring how the individual hardware components communicate and interact. It provides a thorough technical description of the ESP8266 microcontroller, the DHT11 sensor, the I2C LCD, and the transistor-driven actuator circuit. Furthermore, it breaks down the software design and program flow, illustrating the logic utilized to control the hardware, supported by a system block diagram.

Chapter 3: Implementation, Testing, and Conclusion - The third chapter transitions from design theory to physical reality. It describes the practical implementation of the system, including the assembly of the Vero board, wiring configurations, and component housing. It then outlines the testing methodologies used to verify the system's responsiveness to temperature changes. Finally, it provides a concluding summary of the project's success in meeting its objectives and offers recommendations for future iterations and improvements.

Chapter 4: Appendix - The final chapter serves as a technical reference repository. It contains the complete, unedited C++ source code utilized to program the ESP8266

microcontroller, providing full transparency into the software logic driving the automated control system.

Chapter 2: System Design & Architecture

2.1 System Overview

The architecture of the automated temperature and humidity control system is predicated on a straightforward but highly robust closed-loop data processing mechanism. Fundamentally, the system is designed to acquire analog environmental stimuli, convert them into digital data, process that data against defined logical parameters, and execute physical actuation based on the computational results. This workflow is divided into three primary stages: input (sensing), processing (logic execution), and output (display and actuation).

At the center of this architecture is the ESP8266 microcontroller unit (MCU), which orchestrates all communication and timing. The system operates on a continuous, infinitely looping cycle with a programmed 500-millisecond delay to ensure stability and prevent data collision. During each cycle, the MCU requests real-time environmental metrics from the DHT11 sensor via a single-wire serial interface. Once the digital packet containing the current temperature and relative humidity is received, the microcontroller parses this data and temporarily stores it in its volatile memory as floating-point variables.

Following data acquisition, the MCU performs two simultaneous operations. First, it formats the raw sensor data into readable text strings and transmits them over an Inter-Integrated Circuit (I2C) bus to the 16x2 Liquid Crystal Display. This provides the continuous visual output stage of the system. Concurrently, the MCU evaluates the temperature variable against a hardcoded threshold parameter set at 33°C. If the logical condition evaluates to true (meaning the ambient temperature is greater than or equal to 33°C), the MCU triggers its output stage by driving a specific General-Purpose Input/Output (GPIO) pin to a HIGH logic state (3.3V).

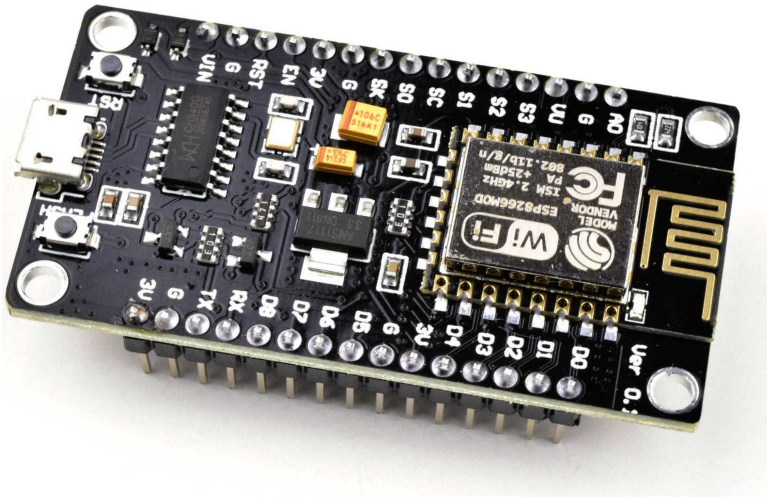
This signal biases the base of an NPN transistor, allowing a larger current to flow through the collector-emitter junction, thereby powering the DC cooling fan. Conversely, if the temperature drops below the threshold, the GPIO pin is pulled LOW (0V), shutting off the transistor and halting the fan.

2.2 Hardware Components and Description

2.2.1 The ESP8266 Microcontroller

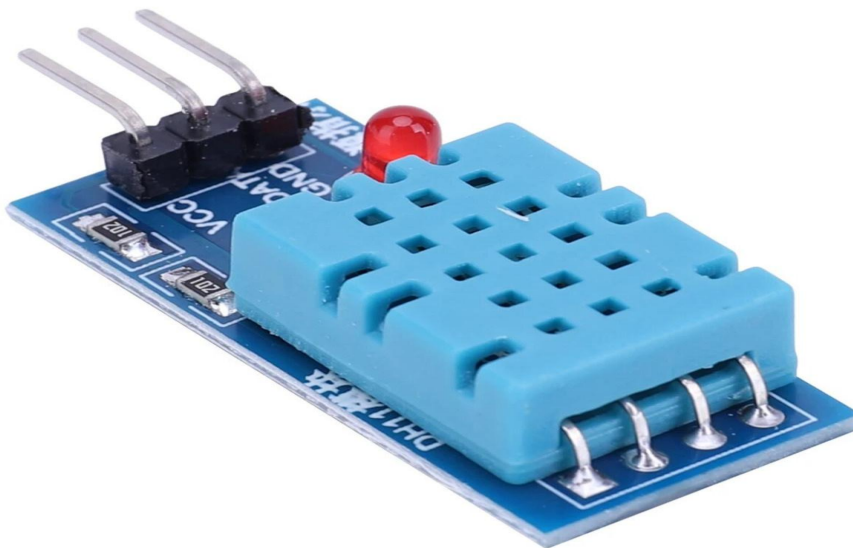
Serving as the central computational brain of the system, the ESP8266 is a low-cost, highly integrated MCU. While it features native Wi-Fi capabilities, this specific project leverages its robust processing power and versatile GPIO mapping for localized control. The board operates fundamentally on 3.3V logic. In this system design, it acts as the master device, generating the clock signals for I2C communication (via pins D1 and D2), receiving digital sensor pulses (via pin D5), and providing the switching logic for the

actuator (via pin D0). It is powered via its onboard micro-USB port, utilizing its internal voltage regulator to distribute appropriate power to the peripheral modules.



2.2.2 DHT11 Temperature and Humidity Sensor

The DHT11 is a basic, ultra-low-cost digital temperature and humidity sensor. It utilizes a capacitive humidity sensor and a thermistor to measure the surrounding air, outputting a digital signal on its data pin. It does not output analog voltages; rather, it contains its own internal analog-to-digital converter (ADC) and an 8-bit microcontroller that formats the readings into a specific digital pulse train. The sensor is rated to measure temperatures from 0°C to 50°C with an accuracy of $\pm 2^{\circ}\text{C}$. Within this system, it is connected to the 3.3V supply of the MCU and transmits its data packets directly to pin D5 upon request.



2.2.3 16x2 Liquid Crystal Display with I2C Interface

To provide localized user feedback, a standard 16x2 character LCD is employed, capable of displaying two distinct rows of sixteen alphanumeric characters each. To mitigate the high pin-count usually required to drive such a display, an I2C 'backpack' module is soldered directly to the display. This module converts the parallel data requirements of the LCD into a two-wire serial bus: Serial Data (SDA) and Serial Clock (SCL). By connecting these to pins D2 and D1 respectively, the ESP8266 can control the entire display using only two pins while powering it from the 5V rail.



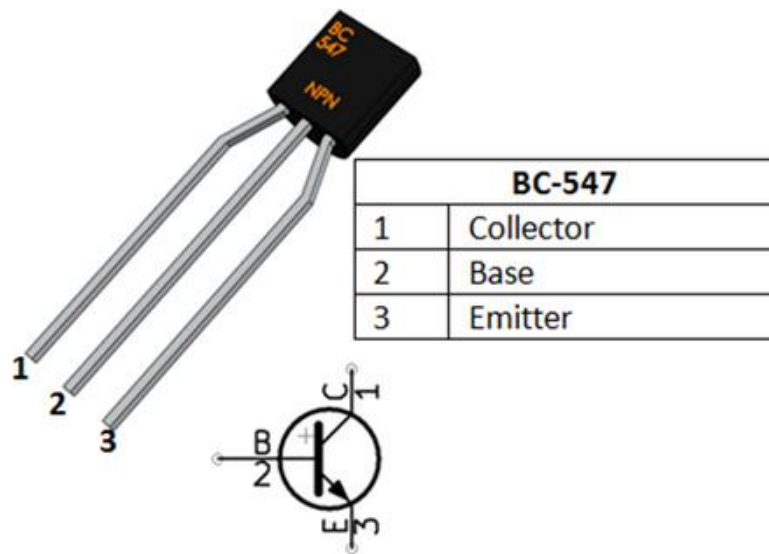
2.2.4 Direct Current (DC) Cooling Fan

The primary actuator of the system is a small, low-voltage DC motor equipped with a propeller blade. This fan is responsible for converting electrical energy into mechanical rotational energy, thereby generating the physical airflow required to lower the ambient temperature in the targeted environment. It is powered by the 5V supply line passed through from the MCU's USB connection.



2.2.5 BC547 NPN Transistor and Current Limiting Resistor

A critical challenge in embedded hardware design is that microcontroller GPIO pins are designed to transmit logic signals, not power. The D0 pin of the ESP8266 can typically source a maximum of roughly 12mA of current, which is vastly insufficient to drive the inductive load of a DC motor. To solve this, a BC547 Bipolar Junction Transistor (BJT) is utilized as a solid-state electronic switch. A 2k ohm resistor is placed in series between the ESP8266's D0 pin and the base of the transistor to limit current draw. When the microcontroller outputs a HIGH signal, the transistor allows current to flow from the power rail, through the fan, and down to the common ground, safely actuating the cooling mechanism.



2.3 Microprocessor Architecture of the ESP8266

The processing core of the system is governed by the ESP8266EX system-on-a-chip (SoC). At its heart, the ESP8266 utilizes a Tensilica L106 Diamond series 32-bit micro-RISC processor. This architecture is engineered for low power consumption while maintaining high computational efficiency, operating natively at a clock frequency of 80 MHz.

One of the defining architectural features leveraged in this project is the SoC's General-Purpose Input/Output (GPIO) multiplexing. In this system design, the internal multiplexer is configured via software to assign the I2C Serial Clock (SCL) and Serial Data (SDA) lines to GPIO 5 and GPIO 4 (mapped as D1 and D2). Furthermore, GPIO 14 (mapped as D5) is configured as a standard digital input to read the proprietary one-wire signal from the DHT11, while GPIO 16 (mapped as D0) is configured as a digital output to drive the transistor logic.

2.4 Software Design and Program Flow

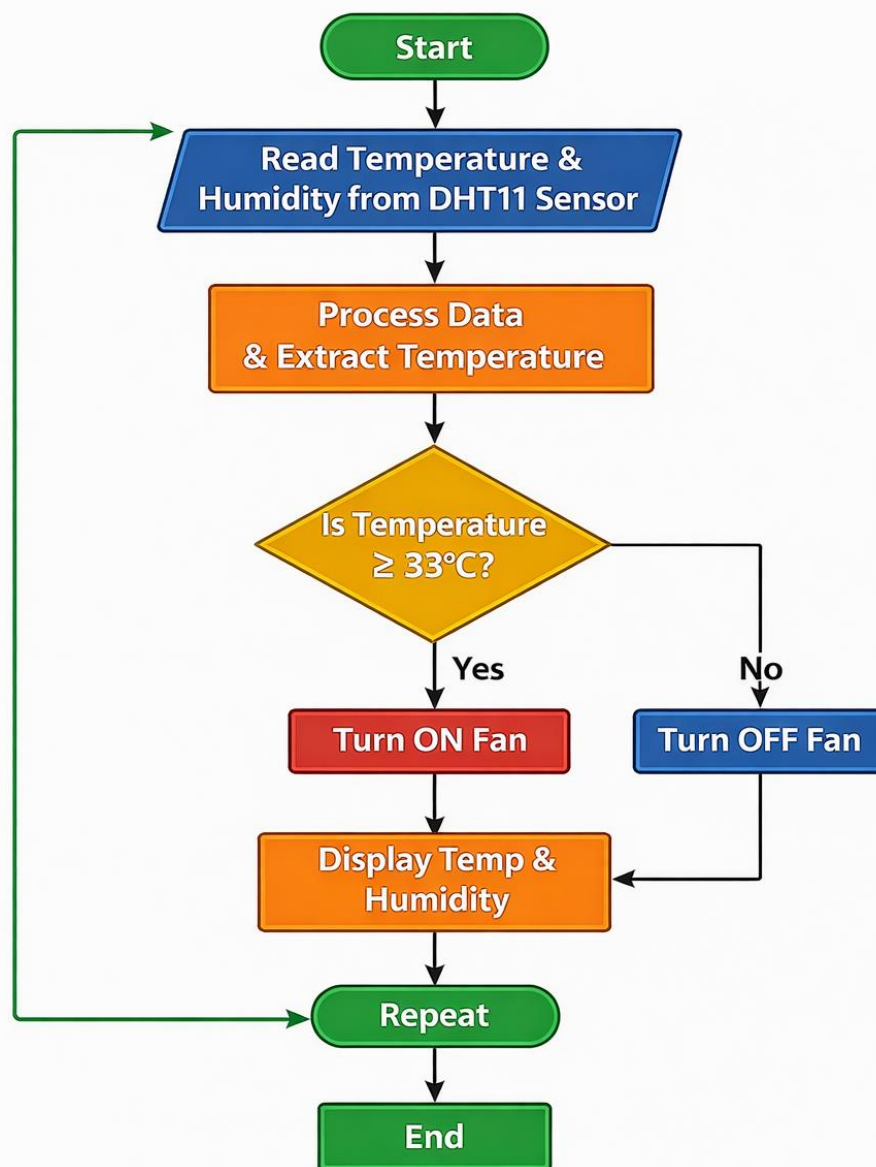
2.4.1 Initialization (The setup() Function)

Upon applying power to the microcontroller, the setup() function executes exactly once. This routine configures the initial state of the hardware by formally assigning the SDA and SCL roles to the respective GPIO pins for I2C communication. The FAN_PIN (D0) is explicitly declared as an OUTPUT, establishing the electrical pathway for the transistor control signal. The LCD is then initialized, and the DHT sensor's internal timing sequence is started.

2.4.2 Operational Logic (The loop() Function)

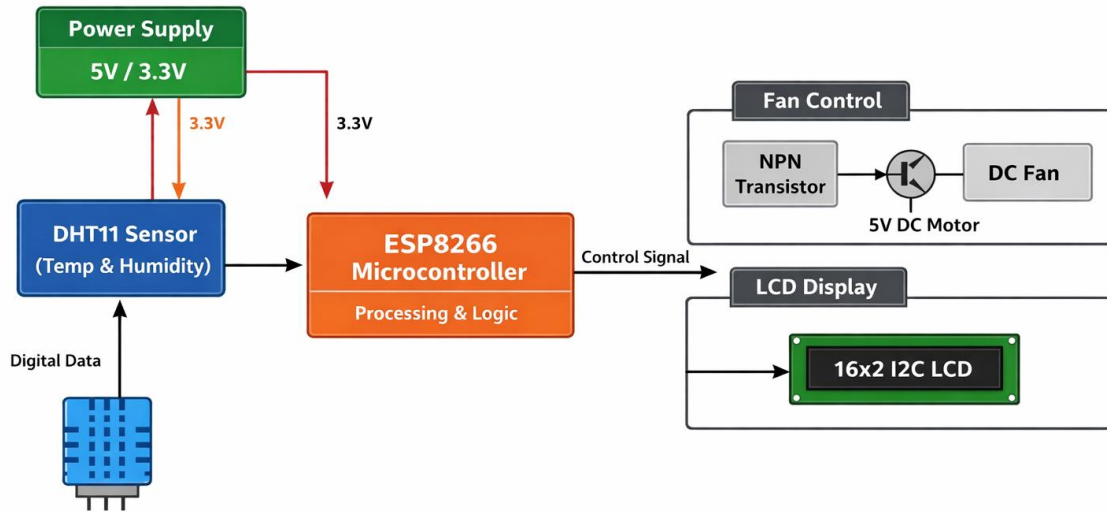
Following initialization, the MCU enters an infinite loop. The microcontroller first calls the sensor functions, halting program execution briefly to await the digital pulse train, and storing the returned values in local floating-point variables. A conditional if-else statement evaluates the physical environment against the 33°C threshold. The LCD screen is subsequently cleared and refreshed with the dynamically updated variable strings. Finally, a delay(500) command pauses execution for 500 milliseconds to prevent overwhelming the I2C bus.

2.4.3 System Program Flowchart



The software flowchart above visually dictates the algorithmic decision-making process embedded within the microcontroller. It highlights the continuous loop structure, where data is sequentially acquired, extracted, and evaluated. The central decision block represents the core intelligence of the system: verifying if the acquired temperature exceeds the threshold.

2.5 System Block Diagram



The system block diagram provides a high-level visual representation of the hardware architecture, illustrating how power, digital data, and control signals flow between the various sub-modules of the automated temperature control system.

Chapter 3: Implementation, Testing, and Conclusion

3.1 System Implementation

3.1.1 Prototype Fabrication and Assembly

The physical implementation of the automated thermal management system involved transitioning from theoretical schematics to robust hardware fabrication. The primary structural component is the white rigid polymer casing, which houses the sensitive internal circuitry while providing external mount points for user interface elements and sensors. Initial modifications to the casing included precise cutouts for the 16x2 Liquid Crystal Display, the micro-USB power input port of the ESP8266, and ventilation entry points to ensure accurate airflow for the internal sub-systems.

The internal electronics were consolidated using a standard phenolic Vero board. This prototyping board served as the structural substrate for soldering, providing permanent, reliable electrical connections and eliminating the risk of loose jumper wires found in initial breadboard testing. The ESP8266 development board was mounted directly to the Vero board using female header pins. This design choice allows for modularity, permitting easy removal of the microcontroller for separate firmware updates if necessary.

3.1.2 Sub-System Integration and Housing

Following the consolidation of the microcontroller core, peripheral modules were integrated. The logic control sub-circuit, utilizing the BC547 Bipolar Junction Transistor (BJT) and the 2k ohm current limiting resistor, was soldered in an emitter-follower configuration near the D0 pin of the MCU. The primary power rail (5V) and common ground (GND) were propagated across the Vero board using 22AWG solid core copper wire to power the peripherals.

The 16x2 LCD display, complete with its pre-soldered I2C backpack module, was securely fastened to the front of the casing. Connections were made using flexible ribbon cables, routed internally to pins D1 (SCL) and D2 (SDA). The external DHT11 Temperature and Humidity sensor module was mounted externally to the casing via a robust three-wire cable assembly (GND, 3.3V, Data). External mounting is critical for ensuring the sensor measures ambient environment variables rather than the localized internal temperature generated by the operational ESP8266.

A unique implementation challenge was the secure mounting of the 5V DC Motor (the fan). This was achieved by utilizing a rigid, blue polymer support rod, which was fixed to the casing's base. The motor unit was then integrated onto this support structure and the output wire junction was insulated using an application of generic thermoplastic adhesive (hot glue) to prevent short circuits and provide strain relief for the delicate motor wires.

3.2 Testing and Results

3.2.1 Test Methodology and Experimental Setup

To validate the system's compliance with the engineering aim and specific technical objectives, a rigorous methodological testing approach was employed. The testing was conducted in a controlled environment. The system was powered via a standard 5V/2A DC USB wall adapter connected to the ESP8266's on-board micro-USB port. The primary goal of testing was to confirm accurate sensor data acquisition, reliable data visualization, and the precise actuation of the cooling fan at the defined 33°C threshold.

The testing protocol involved applying localized, low-intensity heat to the external DHT11 sensor using a standard hair dryer from a distance of roughly 30 cm. The distance was carefully maintained to induce a steady, safe temperature rise without risking thermal damage to the sensor module's plastic casing or the internal processing unit.

3.2.2 Observation of Experimental Results

The initial results confirmed that all peripheral devices booted successfully and synchronized with the ESP8266. Upon system initialization, the LCD activated its backlight and successfully displayed the 'System Starting' message as per the setup() logic in the firmware. After the designated 3000 ms delay, the LCD cleared and began displaying real-time environmental data in a readable 'Temp: XX.XX C' and 'Hum: XX.XX %' format, confirming the successful deployment of the Wire.h and LiquidCrystal_I2C.h libraries.

Scenario A: Ambient Monitoring (Temperature < 33°C) – With the system powered and sitting in a neutral environment, the sensor reported stable room conditions. For example, a reliable data capture from the visualization testing showed environmental conditions of approximately 31.90°C temperature and 52.00% relative humidity. As this reading is below the defined threshold, the ESP8266's logic successfully pulled pin D0 to a LOW (0V) state, and the cooling fan remained stationary.

Scenario B: Triggered Cooling State (Temperature ≥ 33°C) – Upon the application of external heat, the system demonstrated excellent responsiveness. As the localized temperature around the DHT11 rose and crossed the logical gate threshold, the system logic immediately reacted. When the temperature reached approximately 33°C, the firmware instructed D0 to transition to a HIGH state. The resulting 3.3V signal was applied to the base of the BC547 transistor, switching it on and allowing the DC fan to rotate and generate airflow for thermal management.

3.2.3 Analysis of As-Built Performance Flaw (Emitter-Follower)

While the system successfully achieved its goal of turning the fan on at 33°C, an empirical engineering analysis revealed that the fan operated at a visibly lower speed

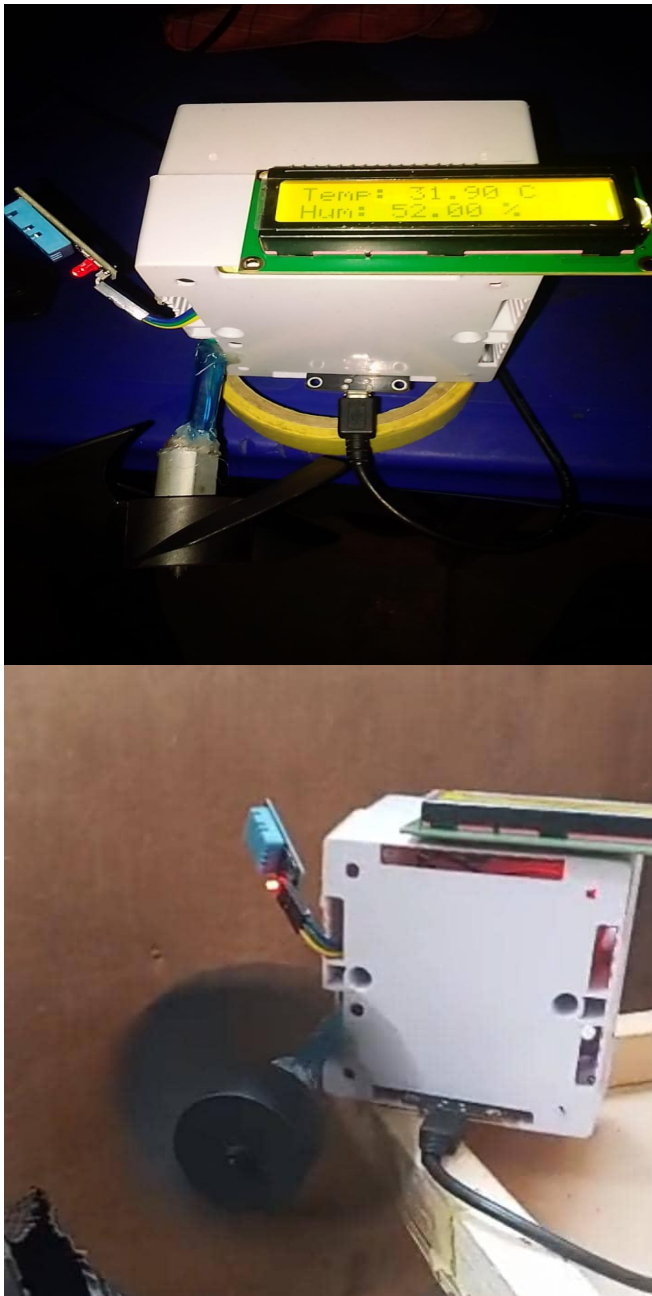
compared to when it was connected directly to a 5V supply. This behavior is attributed to the emitter-follower configuration used in the prototype design.

In an emitter-follower configuration, the transistor provides current gain but introduces a voltage drop of approximately 0.7V between the base and emitter junction. Since the ESP8266 logic output is limited to 3.3V, the effective voltage delivered to the fan is roughly 2.6V (3.3V minus the base-emitter drop). Consequently, the motor receives only about half of its rated voltage, reducing its maximum rotational speed and cooling efficiency. Nevertheless, the prototype still demonstrated the intended switching functionality and control logic effectively for proof-of-concept purposes.

3.3 Conclusion and Future Work

3.3.1 Conclusion

The design and implementation of this microcontroller-based automated environmental control system were successfully completed. The final prototype achieved the primary objectives defined at the beginning of the project by integrating sensing, processing, visualization, and actuation into a cohesive embedded system.



The DHT11 sensor reliably provided environmental measurements, while the I2C LCD displayed the information clearly for the user. The microcontroller's logic successfully evaluated the temperature conditions and controlled the cooling fan when the threshold temperature of 33°C was reached. Despite the voltage limitation caused by the emitter-follower transistor configuration, the prototype successfully demonstrated the fundamental principles of embedded control systems, transistor switching, and hardware-software integration.

3.3.2 Future Work and Recommended Optimizations

Several improvements can be implemented in future iterations of this system to enhance performance, reliability, and functionality.

1. **Transistor Configuration Optimization (Low-Side Switch):** The BC547 transistor should be reconfigured into a common-emitter low-side switching configuration. This would allow the fan to receive the full 5V supply voltage, enabling it to operate at maximum speed and significantly improving cooling performance.
2. **Inductive Load Protection (Flyback Diode):** A flyback diode such as the 1N4007 should be installed across the fan terminals to protect the transistor and microcontroller from back electromotive force (EMF) generated when the motor is switched off.
3. **Advanced Software Control (Hysteresis and Wi-Fi):** Future firmware revisions could implement hysteresis control (for example, fan ON at 33°C and OFF at 31°C) to prevent rapid switching around the threshold temperature. Additionally, the ESP8266's built-in Wi-Fi capability could be utilized to enable remote monitoring through an IoT dashboard or mobile application.

Chapter 4: Appendix

4.1 Source Code

The following section contains the complete C++ source code developed for this project. The firmware was written and compiled utilizing the Arduino Integrated Development Environment (IDE) and flashed to the ESP8266 microcontroller.

The code relies on three primary external libraries: Wire.h for managing the core I2C bus communication, LiquidCrystal_I2C.h for interfacing with the 16x2 display backpack, and the Adafruit DHT.h library for extracting the proprietary digital pulse train from the DHT11 environmental sensor. The logic defines the crucial hardware pinouts and establishes the 33°C threshold that governs the automated actuation of the DC cooling fan.

C++

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <DHT.h>
```

```
// Hardware pin definitions
```

```
#define DHTPIN D5
```

```
#define DHTTYPE DHT11
```

```
#define FAN_PIN D0
```

```
// Create sensor and screen objects
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
// Define the logical thermal threshold
```

```
float thresholdTemp = 33;
```

```
void setup() {  
    Serial.begin(9600);  
    delay(1000);  
  
    // Set D0 (FAN_PIN) as an output to drive the transistor  
    pinMode(D0, OUTPUT);  
  
    // Initialize I2C communication (SDA = D2, SCL = D1)  
    Wire.begin(D2, D1);  
  
    // Initialize the LCD and turn on the backlight  
    lcd.init();  
    lcd.backlight();  
  
    // Start the DHT11 sensor  
    dht.begin();  
    pinMode(FAN_PIN, OUTPUT);  
  
    // Display startup message for visual verification  
    lcd.setCursor(0,0);  
    lcd.print("System Starting");  
    delay(3000);  
    lcd.clear();  
}
```

```
void loop() {  
  
    // Poll the sensor for current environmental data  
  
    float temperature = dht.readTemperature();  
  
    float humidity = dht.readHumidity();  
  
  
    // Control Fan Logic (Threshold Evaluation)  
  
    if (temperature >= thresholdTemp)  
        digitalWrite(FAN_PIN, HIGH);  
    else  
        digitalWrite(FAN_PIN, LOW);  
  
  
    // Display Data on LCD  
  
    lcd.clear();  
  
  
    // Primary Row: Temperature  
  
    lcd.setCursor(0,0);  
  
    lcd.print("Temp: ");  
  
    lcd.print(temperature);  
  
    lcd.print(" C");  
  
  
    // Secondary Row: Humidity  
  
    lcd.setCursor(0,1);  
  
    lcd.print("Hum: ");  
  
    lcd.print(humidity);  
  
    lcd.print(" %");  
}
```

```
// Cycle delay to maintain bus stability  
delay(500);  
}
```